

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN HỌC: TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

**XÂY DỰNG HỆ THỐNG GIAO DỊCH TIỀN MÃ HÓA VỚI TẦNG
META-LABELING BẰNG CATBOOST, THÔNG QUA ĐÓ ĐÁNH
GIÁ ẢNH HƯỞNG CỦA PHƯƠNG PHÁP CHIA DỮ LIỆU ĐẾN ĐỘ
TÍN CẬY CỦA KẾT QUẢ KIỂM ĐỊNH**

Giảng viên hướng dẫn: PGS.TS. Nguyễn Đình Hiền

Lớp: CS106.F31.CN2.TTNT

Nhóm thực hiện: Nhóm 11

TP. Hồ Chí Minh, tháng 9 năm 2026

MỤC LỤC

TÓM TẮT.....	7
CHƯƠNG 1. GIỚI THIỆU VÀ PHÁT BIỂU VẤN ĐỀ.....	8
1.1. Bối cảnh.....	8
1.2. Vấn đề trung tâm: rò rỉ nhãn.....	8
1.3. Câu hỏi nghiên cứu.....	9
1.4. Đóng góp của đồ án.....	10
1.5. Phạm vi và giới hạn chủ động	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	11
2.1. Gán nhãn theo luật triple barrier.....	11
2.2. Tham số gán nhãn đã chốt	12
2.3. Bốn cách chia dữ liệu cho chuỗi thời gian	12
2.3.1. Random K-Fold.....	12
2.3.2. Grouped K-Fold.....	12
2.3.3. Walk-forward	13
2.3.4. Purged Walk-forward kèm embargo	13
2.4. Thuật toán CatBoost.....	14
2.5. Chỉ số đánh giá	14
2.5.1. Nhóm chỉ số phân loại.....	14
2.5.2. Nhóm chỉ số tài chính.....	14
CHƯƠNG 3. DỮ LIỆU VÀ ĐẶC TRƯNG	16
3.1. Nguồn dữ liệu	16
3.2. Sinh tập lệnh ứng viên.....	16
3.3. Niêm phong tập holdout	16
3.4. Bộ 23 đặc trưng đầu vào.....	17
3.5. Các cột bị loại khỏi đầu vào mô hình	18
3.6. Phân bố nhãn theo thời gian	18
CHƯƠNG 4. PHƯƠNG PHÁP THỰC NGHIỆM	20
4.1. Nguyên tắc thiết kế: chỉ thay đổi một biến.....	20
4.2. Bốn cách chia được so sánh	20
4.3. Năm khúc thời gian và việc loại khúc 1	20
4.4. Cấu hình mô hình	21
4.5. Sinh và sử dụng xác suất out-of-fold.....	21
4.6. Quy tắc lọc top-k	21
4.7. Vấn đề tập lệnh ứng viên cố định.....	22
4.8. Quy trình holdout năm giai đoạn.....	22

4.9. Biện pháp bảo đảm tái lập	22
CHƯƠNG 5. KẾT QUẢ THỬ NGHIỆM.....	24
5.1. Chỉ số phân loại của bốn cách chia	24
5.2. Chỉ số tài chính khi giữ top 50%	25
5.3. Độ nhảy theo tỷ lệ giữ lệnh của bốn nhánh	26
5.4. Kết quả trên tập holdout niêm phong	27
5.5. Độ nhảy theo tỷ lệ giữ lệnh trên holdout	27
5.6. Kiểm chứng tái lập	29
5.7. Thí nghiệm có kiểm soát về tham số thread_count	29
CHƯƠNG 6. BÀN LUẬN	31
6.1. Trả lời các câu hỏi nghiên cứu	31
6.1.1. RQ1 — Mức ảnh hưởng của cách chia dữ liệu	31
6.1.2. RQ2 — Sự hội tụ khi siết chặt cách chia	31
6.1.3. RQ3 — Cách chia nào dự báo đúng nhất	31
6.2. Vì sao Grouped K-Fold nằm ở giữa	31
6.3. Kết quả âm ngoài dự kiến: purge và embargo gần như không có tác dụng	32
6.4. Ý nghĩa thực tiễn	32
6.5. Đối chiếu với tài liệu	33
CHƯƠNG 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	34
7.1. Hạn chế	34
7.2. Hướng phát triển.....	34
CHƯƠNG 8. KẾT LUẬN.....	36
TÀI LIỆU THAM KHẢO	37
PHỤ LỤC A. DANH SÁCH ĐẦY ĐỦ 23 ĐẶC TRƯNG	38
PHỤ LỤC B. BẢNG SWEEP ĐẦY ĐỦ CỦA BỐN NHÁNH	39
PHỤ LỤC C. LỊCH SỬ BA PHIÊN BẢN KẾT QUẢ HOLDOUT	41

DANH MỤC BẢNG

Bảng 2.1. Tham số gán nhãn triple barrier đã chốt
Bảng 3.1. Quy mô tập lệnh ứng viên theo giai đoạn
Bảng 3.2. Bộ 23 đặc trưng đầu vào của mô hình
Bảng 3.3. Bảy cột metadata bị loại khỏi đầu vào mô hình
Bảng 3.4. Phân bố nhãn theo năm khúc thời gian
Bảng 4.1. Bốn cách chia dữ liệu được so sánh
Bảng 4.2. Cấu hình CatBoost dùng chung cho cả bốn nhánh
Bảng 5.1. Chỉ số phân loại của bốn cách chia và của holdout
Bảng 5.2. Chỉ số tài chính khi giữ top 50% trên khúc 2–5
Bảng 5.3. Trích bảng sweep tỷ lệ giữ lệnh của bốn nhánh
Bảng 5.4. Kết quả holdout niêm phong khi giữ top 50%
Bảng 5.5. Sweep tỷ lệ giữ lệnh trên tập holdout
Bảng 5.6. Kết quả kiểm chứng tái lập
Bảng 5.7. Ảnh hưởng của thread_count tới kết quả huấn luyện
Bảng 6.1. So sánh ước lượng của từng cách chia với holdout

DANH MỤC HÌNH

Hình 1.1. Cửa sổ nhãn chồng lẫn của bốn lệnh trong cùng một họ pyramid

Hình 2.1. Sáu luật gán nhãn triple barrier như đã cài đặt

Hình 2.2. Bốn cách chia dữ liệu trên cùng một trục thời gian

Hình 3.1. Sơ đồ pipeline tổng thể của hệ thống

Hình 5.1. Đường equity của bốn nhánh trên khúc 2–5 khi giữ top 50%

Hình 5.2. Đường equity trên tập holdout niêm phong khi giữ top 50%

Hình 5.3. Net profit trên holdout theo từng mức tỷ lệ giữ lệnh

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Tiếng Việt / Giải thích
Label leakage	Rò rỉ nhãn — thông tin của tập kiểm tra lọt vào tập huấn luyện khiến điểm số bị thổi phồng
Meta-labeling	Kiến trúc hai tầng: mô hình sơ cấp quyết định vào lệnh, mô hình thứ cấp chỉ đánh giá tín hiệu đó có đáng tin không
Backtest overfitting	Thổi phồng kết quả kiểm định do thử nhiều cấu hình rồi chỉ báo cáo cấu hình đẹp nhất
Triple barrier	Gán nhãn ba hàng rào: chặn trên (chốt lời), chặn dưới (cắt lỗ), chặn thời gian
Vertical barrier	Chặn thời gian — giới hạn số bar tối đa một lệnh được giữ
Pyramid / leg	Chiến lược chồng lệnh; mỗi tín hiệu gốc mở 1 lệnh nền và 3 lệnh chồng (leg 0–3)
origin_bar	Mã bar sinh ra tín hiệu gốc — định danh của một họ lệnh gồm 4 leg
K-Fold	Kiểm định chéo chia dữ liệu thành K phần
Grouped K-Fold	K-Fold có ràng buộc: các mẫu cùng nhóm luôn nằm cùng một phía
Walk-forward	Kiểm định tiến theo thời gian: train quá khứ, test tương lai
Purging	Loại các mẫu huấn luyện có cửa sổ nhãn chồng lấn với tập kiểm tra
Embargo	Vùng đệm cấm thêm sau biên tập kiểm tra
OOF (out-of-fold)	Dự báo cho mẫu ở fold mà mô hình không được huấn luyện trên đó
Holdout	Tập dữ liệu niêm phong, tách trước khi huấn luyện và chỉ mở một lần để đối chứng
R	Bội số rủi ro — lãi/lỗ của một lệnh tính theo bội số khoảng cắt lỗ ban đầu
MaxDD	Maximum drawdown — mức sụt giảm sâu nhất của đường vốn
Profit factor (PF)	Tổng lãi chia tổng lỗ; $PF > 1$ nghĩa là có lãi ròng
ROC-AUC	Diện tích dưới đường ROC — đo khả năng xếp hạng của mô hình phân loại
ATR	Average True Range — thước đo biên độ dao động trung bình

TÓM TẮT

Đồ án này không nhằm xây dựng một hệ thống giao dịch sinh lời, mà nhằm **đo lường mức sai lệch mà một quyết định phương pháp luận duy nhất — cách chia dữ liệu — có thể gây ra** khi đánh giá mô hình học máy trên chuỗi thời gian tài chính có nhãn chồng lấn.

Đối tượng nghiên cứu là một chiến lược pyramid trên BTCUSD khung M15: mỗi tín hiệu gốc mở một lệnh nền và ba lệnh chồng, tạo thành họ bốn lệnh có cửa sổ nhãn triple barrier chồng lấn gần như hoàn toàn. Mô hình CatBoost đóng vai trò **tầng meta-labeling** đặt chồng lên chiến lược sơ cấp: nó không quyết định vào lệnh mà chỉ chấm điểm từng lệnh ứng viên để loại bớt lệnh kém. Trên 25.008 lệnh trước mốc niêm phong, chúng tôi huấn luyện cùng một mô hình với cùng một bộ siêu tham số theo bốn cách chia dữ liệu có độ chặt tăng dần: Random K-Fold, Grouped K-Fold, Walk-forward và Purged Walk-forward. Toàn bộ kết quả sau đó được đối chứng trên một tập holdout 5.028 lệnh đã niêm phong từ trước khi huấn luyện.

Kết quả cho thấy điểm số suy giảm đơn điệu theo độ chặt của cách chia: ROC-AUC lần lượt là $0,8582 \rightarrow 0,7454 \rightarrow 0,5875 \rightarrow 0,5895$, trong khi holdout đạt 0,6046 — tức **Random K-Fold thổi phồng ROC-AUC khoảng 0,25 đơn vị tuyệt đối**. Trên chỉ số tài chính, chênh lệch còn lớn hơn: cùng một tỷ lệ giữ lệnh 50%, nhánh Random K-Fold báo lãi +6.937,53 R còn nhánh Purged Walk-forward chỉ báo +2.119,48 R. Trên holdout, bộ lọc **làm xấu đi** kết quả so với việc không lọc (−36,55 R so với +245,93 R) và thứ hạng xác suất còn bị đảo chiều: lọc càng gắt thì kết quả càng âm.

Đóng góp chính của đồ án là một quy trình đánh giá khép kín và tái lập được, kèm bằng chứng kiểm chứng từng byte, cho phép định lượng chính xác phần điểm số đến từ rò rỉ nhãn thay vì từ năng lực dự báo thật của mô hình.

Từ khóa: meta-labeling, rò rỉ nhãn, triple barrier, walk-forward, purging, embargo, CatBoost, holdout niêm phong, tái lập kết quả.

CHƯƠNG 1. GIỚI THIỆU VÀ PHÁT BIỂU VẤN ĐỀ

1.1. Bối cảnh

Học máy được áp dụng rộng rãi vào dữ liệu thị trường tài chính, nhưng phần lớn công cụ đánh giá tiêu chuẩn của ngành học máy lại được thiết kế cho dữ liệu thỏa giả định các mẫu độc lập và cùng phân phối. Chuỗi thời gian tài chính vi phạm giả định này theo nhiều cách, và một trong những cách nguy hiểm nhất là **nhãn của các mẫu chồng lấn nhau về thời gian**.

Đối tượng cụ thể của đề án là một chiến lược giao dịch dạng pyramid trên cặp BTCUSD, khung thời gian M15. Khi một tín hiệu vào lệnh xuất hiện tại một bar, chiến lược mở một lệnh nền và tuần tự mở thêm ba lệnh chồng khi giá đi đúng hướng. Như vậy mỗi tín hiệu gốc sinh ra **bốn lệnh cùng một `origin\_bar`**, gọi là một họ lệnh.

Vai trò của mô hình học máy trong hệ thống này cần được nói rõ ngay từ đầu: CatBoost **không quyết định vào lệnh**. Chiến lược pyramid vẫn là bên sinh ra toàn bộ tín hiệu; mô hình chỉ chấm điểm từng lệnh ứng viên và những lệnh điểm thấp sẽ bị bỏ qua. Nói cách khác, mô hình là một **bộ lọc tín hiệu** đặt sau chiến lược, không phải một bộ ra quyết định độc lập.

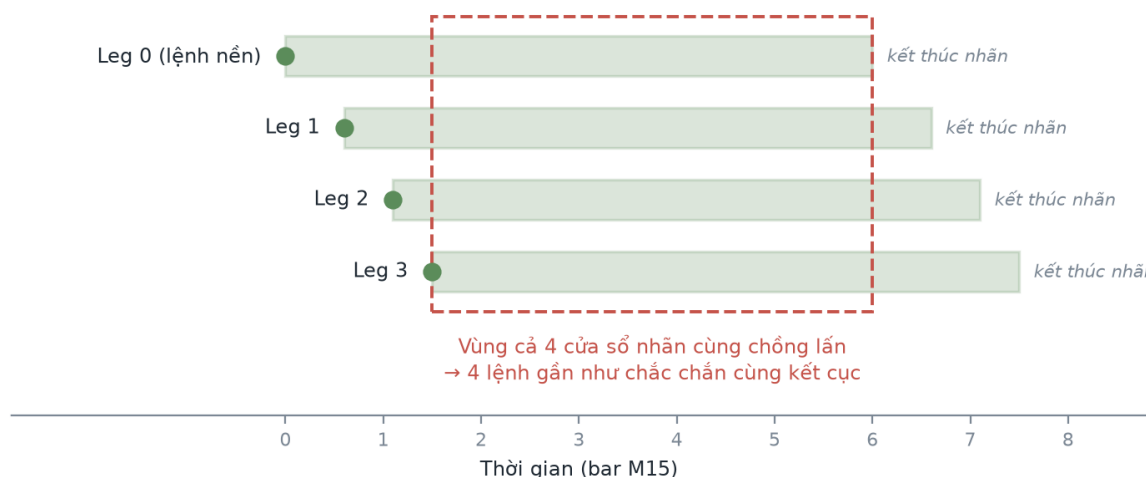
Cách bố trí này chính là **meta-labeling** theo thuật ngữ của López de Prado [1]: một mô hình sơ cấp (chiến lược pyramid) quyết định *có vào lệnh hay không và vào theo hướng nào*, còn một mô hình thứ cấp đặt chồng lên trên chỉ trả lời một câu hỏi nhị phân hẹp hơn — *tín hiệu này có đáng tin không*. Ưu điểm của kiến trúc hai tầng là mô hình thứ cấp không phải học lại việc dự báo hướng giá, vốn là phần khó nhất; nó chỉ cần học cách phân biệt tín hiệu tốt với tín hiệu xấu trong tập tín hiệu đã cho. Toàn bộ đề án đo **độ tin cậy của phép đánh giá tầng meta-labeling này**, chứ không đo chất lượng của chiến lược sơ cấp.

1.2. Vấn đề trung tâm: rò rỉ nhãn

Nhãn của mỗi lệnh được gán theo luật triple barrier: quan sát diễn biến giá sau thời điểm vào lệnh và ghi nhận hàng rào nào bị chạm trước — chặn trên (thắng), chặn dưới (thua) hay chặn thời gian. Điều này có nghĩa là **nhãn được xác định bằng cách nhìn về tương lai** của mẫu, trải dài trên một cửa sổ tối đa 50 bar M15.

Từ đó phát sinh hai tầng chồng lấn:

- **Chồng lấn trong cùng họ lệnh.** Bốn leg của một họ mở cách nhau vài bar nhưng cửa sổ nhân kéo dài tới 50 bar, nên chúng gần như luôn cùng thắng hoặc cùng thua.
- **Chồng lấn giữa các họ gần nhau.** Hai họ lệnh mở cách nhau vài chục bar vẫn có cửa sổ nhân giao nhau, nên nhân của chúng tương quan mạnh.



Hình 1.1. Cửa sổ nhân chồng lấn của bốn lệnh trong cùng một họ pyramid

Hậu quả trực tiếp: nếu chia dữ liệu **ngẫu nhiên** thành tập huấn luyện và tập kiểm tra, các lệnh anh em trong cùng một họ sẽ bị đặt ở cả hai phía. Mô hình khi đó không cần học quy luật thị trường — nó chỉ cần nhận ra rằng mẫu đang xét rất giống một mẫu nó đã thấy trong tập huấn luyện, và sao chép đáp án. Điểm số thu được **đo khả năng ghi nhớ chứ không đo khả năng tổng quát hóa**.

Vấn đề này đặc biệt khó phát hiện vì nó **không gây lỗi**. Mã chạy đúng, chỉ số đẹp, biểu đồ thuyết phục. Chỉ khi đưa mô hình ra dữ liệu thật sự mới, sai lệch mới bộc lộ — và lúc đó chi phí đã phát sinh.

1.3. Câu hỏi nghiên cứu

Đề án đặt ra ba câu hỏi có thể kiểm chứng bằng số liệu:

1. **RQ1.** Cách chia dữ liệu ảnh hưởng bao nhiêu tới chỉ số phân loại và chỉ số tài chính, khi mọi yếu tố khác được giữ nguyên?

2. **RQ2.** Khi siết dần độ chặt của cách chia, điểm số có hội tụ về một mức ổn định không?
3. **RQ3.** Trên tập holdout niêm phong chưa từng được nhìn, cách chia nào cho ước lượng sát thực tế nhất?

1.4. Đóng góp của đồ án

- Thiết kế một **thí nghiệm có kiểm soát** so sánh bốn cách chia dữ liệu trên cùng một tập lệnh cố định, cùng bộ đặc trưng và cùng siêu tham số, nhờ đó mọi chênh lệch quan sát được đều quy được về cách chia.
- Sử dụng một **tập holdout niêm phong** tách ra trước mọi bước huấn luyện, đóng vai trò trọng tài độc lập cho cả bốn ước lượng.
- Cung cấp **bộ artifact tái lập được** kèm kiểm chứng từng byte giữa hai lần chạy độc lập và bộ kiểm thử tự động.
- Ghi nhận một **thí nghiệm có kiểm soát ngoài dự kiến** về ảnh hưởng của tham số kỹ thuật `thread_count` tới kết quả huấn luyện CatBoost.

1.5. Phạm vi và giới hạn chủ động

Đồ án **không** nhằm chứng minh chiến lược pyramid sinh lời, **không** đề xuất tham số giao dịch tối ưu và **không** hứa hẹn kết quả trên dữ liệu tương lai. Kết quả holdout ở Chương 5 là kết quả âm; chúng tôi trình bày nguyên trạng vì chính nó là bằng chứng mạnh nhất cho luận điểm trung tâm của đồ án.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Gán nhãn theo luật triple barrier

Phương pháp triple barrier gán nhãn cho một lệnh bằng cách dựng ba hàng rào kể từ thời điểm vào lệnh và xét hàng rào nào bị chạm trước:

- **Chặn dưới** — mức cắt lỗ. Chạm trước thì nhãn là 0 (thua).
- **Chặn trên** — mức chốt lời. Chạm trước thì nhãn là 1 (thắng).
- **Chặn thời gian (vertical barrier)** — số bar tối đa được giữ lệnh. Hết hạn mà chưa chạm hàng rào nào thì nhãn được quyết theo luật đóng lệnh của chiến lược.

So với cách gán nhãn theo lợi suất cố định sau N bar, triple barrier phản ánh sát hơn cách một lệnh thực sự kết thúc trên thị trường, vì nó tôn trọng đồng thời cả mục tiêu lợi nhuận, ngưỡng chịu lỗ và giới hạn thời gian.

Thông số Triple Barrier gán nhãn — Pyramid Strategy (BTC M15)	
Thông số	Giá trị
Upper barrier	$\text{entry_price} + 3.8 \times \text{ATR}(90)$ — ATR(90) lấy tại bar M15 ngay trước bar entry (bar i-1), tính ra một mức giá cố định suốt vòng đời ngay lúc vào lệnh
Lower barrier	$\text{entry_price} \times (1 - 0.006)$ — tức 0.6% dưới entry
Vertical barrier	50 bar M15

Quy tắc gán nhãn	
Duyệt từng bar M15 (tối đa 50 bar), trong mỗi bar duyệt 15 cây M1 theo thứ tự thời gian:	
1. Cùng 1 cây M1 chọc cả upper lẫn lower → label 0, dừng	
2. Low M1 ≤ lower barrier → label 0, dừng (chạm là chết)	
3. High M1 ≥ upper barrier → ghi nhận, chưa gán nhãn, tiếp tục duyệt hết các cây M1 còn lại trong bar M15 đó	
4. Hết bar M15 mà stop chưa bị chạm + đã ghi nhận High ≥ upper → label 1, dừng	
5. Hết bar M15 mà chưa chạm barrier nào → qua bar M15 tiếp theo, lặp lại	
6. Hết 50 bar M15 → label 0	

Lưu ý	
• Ranh giới holdout: 2025-02-08 15:30 — chỉ gán nhãn 80% dữ liệu đầu, không chạm holdout	

Hình 2.1. Sáu luật gán nhãn triple barrier như đã cài đặt trong `labeling/triple_barrier.py`

2.2. Tham số gán nhãn đã chốt

Bảng 2.1. Tham số gán nhãn triple barrier đã chốt

Tham số	Giá trị	Vai trò trong thiết kế
Cắt lỗ (SL)	0,6 %	Định nghĩa đơn vị R; đủ rộng để không bị nhiễu vì cấu trúc quét, đủ hẹp để giữ số lệnh lớn
Chặn trên	$3,8 \times \text{ATR}(90)$	Chuẩn hóa mục tiêu lợi nhuận theo biến động hiện hành thay vì cố định theo phần trăm
Chặn thời gian H	50 bar M15 ($\approx 12,5$ giờ)	Giới hạn độ dài cửa sổ nhãn, đồng thời là độ dài vùng embargo ở Cách 3
Số leg pyramid k	3 (tổng 4 lệnh mỗi họ)	Tạo ra cấu trúc họ lệnh — chính là nguồn rò rỉ mà đồ án đo
Độ phân giải mô phỏng	M1	Xác định thứ tự chạm hàng rào trong cùng một bar M15

Nguồn: docs/bao_cao_hyperparameter_triple_barrier.md

Điểm cần nhấn mạnh: chặn thời gian H = 50 bar vừa là tham số gán nhãn, vừa là **độ dài cửa sổ chòng lún tối đa** giữa hai mẫu bất kỳ. Con số này quyết định trực tiếp độ rộng vùng purge và embargo ở Cách 3, và như Chương 6 sẽ phân tích, nó cũng giải thích vì sao purge/embargo gần như không thay đổi kết quả trong thiết lập này.

2.3. Bốn cách chia dữ liệu cho chuỗi thời gian

2.3.1. Random K-Fold

Chuẩn mực của học máy tổng quát: xáo trộn toàn bộ dữ liệu rồi chia đều thành K phần, lần lượt lấy một phần làm tập kiểm tra. Cách này giả định các mẫu độc lập. Với dữ liệu có nhãn chòng lún, giả định đó sai, và các lệnh anh em sẽ nằm ở cả hai phía — đây chính là kịch bản rò rỉ tối đa mà đồ án dùng làm mốc so sánh.

2.3.2. Grouped K-Fold

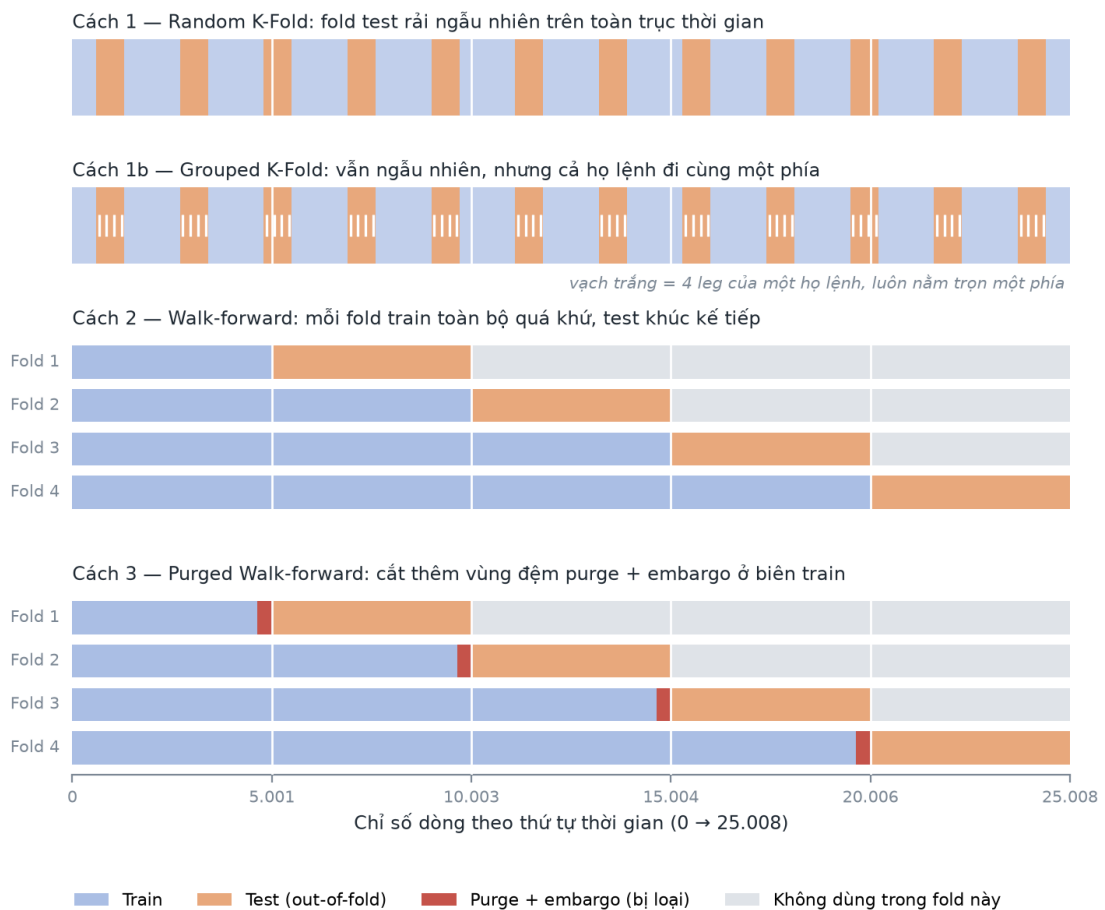
Vẫn chia ngẫu nhiên, nhưng thêm ràng buộc: các mẫu cùng một nhóm phải nằm trọn về một phía. Ở đây nhóm được định nghĩa theo `origin_bar`, nên cả bốn leg của một họ lệnh luôn cùng ở tập huấn luyện hoặc cùng ở tập kiểm tra. Cách này chặn được tầng rò rỉ thứ nhất, **nhưng không chặn được tầng thứ hai** — hai họ lệnh gần nhau về thời gian vẫn có thể bị tách hai phía.

2.3.3. Walk-forward

Tôn trọng mũi tên thời gian: dữ liệu được chia thành các khúc liên tiếp, mô hình luôn được huấn luyện trên toàn bộ quá khứ và kiểm tra trên khúc kế tiếp. Không có mẫu tương lai nào lọt vào tập huấn luyện, nên hai tầng rò rỉ ở trên bị loại bỏ gần hết — chỉ còn phần chồng lấn hẹp ngay tại biên giữa hai khúc.

2.3.4. Purged Walk-forward kèm embargo

Bổ sung cho walk-forward hai cơ chế do López de Prado [1] đề xuất: **purging** loại khỏi tập huấn luyện những mẫu có cửa sổ nhãn còn chồng lấn với tập kiểm tra, và **embargo** cắt thêm một vùng đệm cố định ngay trước biên. Trong đồ án này, độ dài embargo được đặt bằng đúng $H = 50$ bar. Đây là cách chia chặt nhất trong bốn cách được so sánh.



Hình 2.2. Bốn cách chia dữ liệu trên cùng một trục thời gian

2.4. Thuật toán CatBoost

CatBoost [2] là một thư viện gradient boosting trên cây quyết định. Mô hình được xây dựng tuần tự: mỗi cây mới học phần dư của tổ hợp các cây trước đó. Hai đặc điểm khiến CatBoost phù hợp với bài toán này:

- **Ordered boosting** — cơ chế do nhóm tác giả CatBoost [2] đề xuất nhằm giảm thiên lệch dự báo (prediction shift) do rò rỉ mục tiêu trong quá trình boosting, vốn là điểm yếu của gradient boosting cổ điển.
- **Xử lý tốt dữ liệu bảng và mất cân bằng lớp** — tỷ lệ nhãn 1 trong tập dữ liệu chỉ khoảng 26,2 %, nên tham số `auto_class_weights=Balanced` được bật để cân bằng trọng số lớp.

Cần phân biệt rõ hai loại rò rỉ: ordered boosting xử lý rò rỉ **bên trong thuật toán boosting**, còn rò rỉ nhãn do chồng lấn thời gian mà đề án nghiên cứu là **vấn đề ở tầng thiết kế thí nghiệm**. Không thuật toán nào tự khắc phục được loại thứ hai; chỉ cách chia dữ liệu mới làm được.

2.5. Chỉ số đánh giá

Đề án dùng đồng thời hai nhóm chỉ số, vì mỗi nhóm trả lời một câu hỏi khác nhau.

2.5.1. Nhóm chỉ số phân loại

- **ROC-AUC** — xác suất mô hình xếp một mẫu dương ngẫu nhiên cao hơn một mẫu âm ngẫu nhiên. Đo **năng lực xếp hạng**, không phụ thuộc ngưỡng cắt.
- **F1 tại ngưỡng 0,5** — trung bình điều hòa của precision và recall, đo chất lượng phân loại tại một ngưỡng cụ thể.

2.5.2. Nhóm chỉ số tài chính

- **R** — đơn vị bội số rủi ro. Một lệnh lãi đúng bằng khoảng cắt lỗ ban đầu được tính là +1 R. Đơn vị này cho phép cộng dồn các lệnh có quy mô khác nhau.
- **Net profit (R)** — tổng lãi lỗ ròng của toàn bộ lệnh.
- **MaxDD (R)** — mức sụt giảm sâu nhất của đường vốn tính từ đỉnh trước đó.
- **Profit factor** — tổng lãi chia tổng lỗ. $PF > 1$ nghĩa là có lãi ròng.
- **Win rate (%)** — tỷ lệ lệnh có lãi.

Lý do phải dùng cả hai nhóm: một mô hình có thể cải thiện ROC-AUC mà vẫn làm xấu kết quả giao dịch, nếu phần cải thiện rơi vào những lệnh có biên độ lãi lỗ nhỏ. Ngược lại, chỉ số tài chính đơn lẻ dễ bị chi phối bởi vài lệnh biên độ lớn. Chương 5 sẽ cho thấy hai nhóm chỉ số này **không luôn đồng thuận**.

CHƯƠNG 3. DỮ LIỆU VÀ ĐẶC TRƯNG

3.1. Nguồn dữ liệu

Dữ liệu gốc là chuỗi nến BTCUSD khung M1, trải từ **2018-01-01 22:01** đến **2026-08-21 09:33**, dung lượng khoảng 209 MB. Dữ liệu M1 được tổng hợp lên khung M15 để sinh tín hiệu và đặc trưng, nhưng khung M1 vẫn được giữ lại để mô phỏng backtest, vì chỉ ở độ phân giải M1 mới xác định được hàng rào nào bị chạm trước khi cả hai cùng nằm trong một bar M15.

3.2. Sinh tập lệnh ứng viên

Chiến lược pyramid được replay trên toàn bộ lịch sử M1, sinh ra tập lệnh ứng viên cố định. Đây là bước quan trọng về mặt phương pháp: **tập lệnh này được sinh một lần và giữ nguyên cho cả bốn nhánh**, nên mọi chênh lệch kết quả về sau không thể đến từ việc các nhánh nhìn thấy những lệnh khác nhau.

Bảng 3.1. Quy mô tập lệnh ứng viên theo giai đoạn

Giai đoạn	Số lệnh	Ghi chú
Trước mốc holdout	25.008	6.252 họ lệnh $\times$ 4 leg — tập dùng để huấn luyện và so sánh bốn cách chia
Lệnh biên	4	Mở trước mốc nhưng đóng sau mốc; không thuộc tập nào
Trong holdout	5.028	Tập niêm phong, chỉ mở một lần để đối chứng
Tổng toàn lịch sử	30.040	$25.008 + 4 + 5.028$

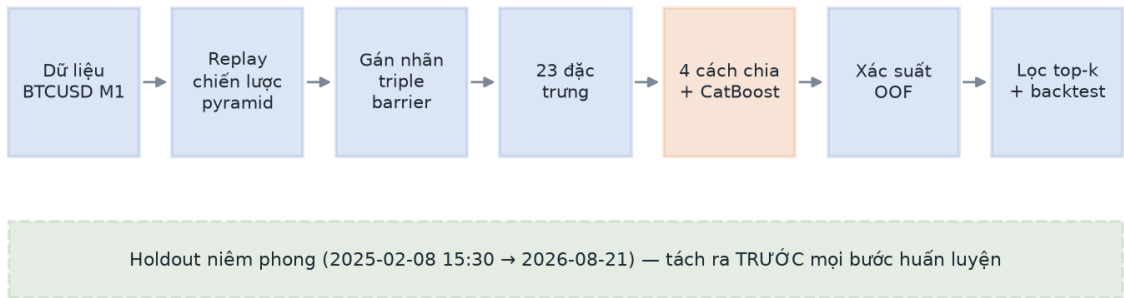
Nguồn: `outputs/holdout/stage3/stage3_report.json`

3.3. Niêm phong tập holdout

Mốc niêm phong được đóng băng tại **2025-02-08 15:30:00**, tương ứng bar M15 thứ 199.968. Mọi lệnh có `entry_time` hoặc `label_end_time` từ mốc này trở đi được tách ra thành tập holdout và **không được đụng tới trong suốt quá trình phát triển**.

Đây là điểm thiết kế quyết định tính hợp lệ của toàn bộ kết luận trong báo cáo. Nếu holdout được tách sau khi đã thử nghiệm, mọi quyết định thiết kế trước đó đều có thể đã vô tình được điều chỉnh theo dữ liệu holdout, và vai trò trọng tài độc lập sẽ mất. Kịch bản kiểm tra tự động (`scripts/verify_dataset.py`) xác nhận điều kiện niêm phong

trước mỗi lần huấn luyện: giá trị `entry_time` lớn nhất trong tập huấn luyện là 2025-02-06 15:45 và `label_end_time` lớn nhất là 2025-02-06 16:44, đều nằm trước mốc.



Hình 3.1. Sơ đồ pipeline tổng thể của hệ thống

3.4. Bộ 23 đặc trưng đầu vào

Mỗi lệnh ứng viên được mô tả bằng 23 đặc trưng số, tính **tại thời điểm vào lệnh** và chỉ dùng thông tin có sẵn đến thời điểm đó. Các đặc trưng được nhóm theo năm chủ đề.

Bảng 3.2. Bộ 23 đặc trưng đầu vào của mô hình

Nhóm	Đặc trưng	Ý nghĩa
A. Hình học hàng rào	<code>breakeven_R</code>	Khoảng cách tới điểm hòa vốn, tính theo bội số R — quyết định độ khó của nhãn
B. Chế độ biến động	<code>atr14_pct</code> , <code>atr_ratio_14_90</code> , <code>vol20</code> , <code>vol200</code> , <code>vol_ratio_20_200</code> , <code>range_pct</code>	Mức biến động hiện hành và tương quan giữa biến động ngắn hạn với dài hạn
C. Xu hướng và vị trí trong biên độ	<code>dist_ema20_atr</code> , <code>dist_ema50_atr</code> , <code>dist_ema200_atr</code> , <code>ret20_atr</code> , <code>ret50_atr</code> , <code>pos_in_range50</code> , <code>dist_hh20_atr</code>	Khoảng cách tới các đường trung bình và tới đỉnh gần nhất, đều chuẩn hóa theo ATR để so sánh được giữa các chế độ biến động
D. Tín hiệu của chính chiến lược	<code>mom14</code> , <code>mom_diff</code> , <code>vwap_dist_atr</code> , <code>vwap_slope_atr</code> , <code>rsi14</code>	Các chỉ báo mà chiến lược dùng để sinh tín hiệu, được tính lại tươi tại thời điểm vào lệnh
E. Khối lượng, vi cấu trúc, phiên	<code>vol_ratio_volume</code> , <code>gap_open_atr</code> , <code>hour</code> , <code>dow</code>	Bối cảnh thanh khoản, khoảng nhảy giá mở cửa và thời điểm trong tuần

Nguồn: `docs/bao_cao_feature.md`; `src/citd_ml/features/build_features.py`

3.5. Các cột bị loại khỏi đầu vào mô hình

Ngoài 23 đặc trưng, tập dữ liệu còn 7 cột metadata. Những cột này **tuyệt đối không được đưa vào mô hình**, và lý do liên quan trực tiếp tới chủ đề của đề án.

Bảng 3.3. Bảy cột metadata bị loại khỏi đầu vào mô hình

Cột	Vai trò	Vì sao không được đưa vào mô hình
entry_time	Thời điểm vào lệnh	Dùng để sắp xếp và chia khúc thời gian
label_end_time	Thời điểm nhận kết thúc	Dùng để tính vùng purge; bản thân nó là thông tin tương lai
origin_bar	Định danh họ lệnh	Khóa nhóm cho Grouped K-Fold
entry_bar	Chỉ số bar vào lệnh	Dùng để đối chiếu với tradelist
entry_price	Giá vào lệnh	Dùng để tính lãi lỗ khi backtest
`leg`	Thứ tự leg trong họ (0–3)	Mô tả giàn giáo pyramid, không mô tả thị trường. Đưa vào là cho mô hình biết trực tiếp cấu trúc họ lệnh
`entry_vs_base_R`	Khoảng cách tới lệnh nền	Suy ra được vị trí của lệnh trong họ , tương đương rò rỉ cấu trúc nhóm

Hai cột leg và entry_vs_base_R là ví dụ điển hình của rò rỉ tinh vi: chúng có sẵn tại thời điểm vào lệnh nên **không vi phạm nguyên tắc nhân quả thời gian**, và vì vậy rất dễ bị đưa vào một cách vô ý. Nhưng chúng tiết lộ cấu trúc họ lệnh, tức chính là biến mà toàn bộ thí nghiệm đang muốn kiểm soát. Kịch bản kiểm tra pre_train.py chặn cứng điều này: nếu FEATURES và META giao nhau, chương trình dừng với lỗi.

3.6. Phân bố nhãn theo thời gian

Tập 25.008 dòng được chia thành năm khúc liên tiếp theo thời gian, với biên [0, 5001, 10003, 15004, 20006, 25008]. Tỷ lệ nhãn 1 toàn cục là khoảng 26,2 %, nhưng phân bố theo khúc cho thấy dữ liệu **không dừng**.

Bảng 3.4. Phân bố nhãn theo năm khúc thời gian

Khúc	Chỉ số dòng	Số dòng	Khoảng thời gian	Tỷ lệ nhãn 1
1	0 – 5.000	5.001	2018-01-04 → 2019-12-13	27,63 %
2	5.001 – 10.002	5.002	2019-12-13 → 2021-03-28	25,73 %
3	10.003 – 15.003	5.001	2021-03-28 → 2022-07-05	23,92 %

Khúc	Chỉ số dòng	Số dòng	Khoảng thời gian	Tỷ lệ nhãn 1
4	15.004 – 20.005	5.002	2022-07-05 → 2023-10-19	28,61 %
5	20.006 – 25.007	5.002	2023-10-19 → 2025-02-06	24,91 %

Nguồn: scripts/verify_dataset.py

Biên độ dao động gần 5 điểm phần trăm giữa khúc 3 và khúc 4 là bằng chứng cho thấy quan hệ giữa đặc trưng và nhãn thay đổi theo thời gian. Đây là lý do độc lập thứ hai — bên cạnh rò rỉ nhãn — khiến Random K-Fold cho kết quả lạc quan: khi trộn ngẫu nhiên, mô hình luôn được học trên dữ liệu cùng thời kỳ với dữ liệu nó phải dự báo.

CHƯƠNG 4. PHƯƠNG PHÁP THỰC NGHIỆM

4.1. Nguyên tắc thiết kế: chỉ thay đổi một biến

Toàn bộ thí nghiệm được thiết kế quanh một nguyên tắc: **chỉ duy nhất cách chia dữ liệu được thay đổi**. Tập lệnh ứng viên, bộ 23 đặc trưng, nhãn, siêu tham số mô hình, hạt giống ngẫu nhiên và quy tắc lọc đều được giữ nguyên tuyệt đối giữa bốn nhánh.

Nhờ ràng buộc này, mọi chênh lệch quan sát được ở Chương 5 đều quy được về đúng một nguyên nhân. Nếu thay đổi đồng thời hai yếu tố, kết quả sẽ không diễn giải được, và toàn bộ giá trị của đề án sẽ mất.

4.2. Bốn cách chia được so sánh

Bảng 4.1. Bốn cách chia dữ liệu được so sánh

Nhánh	Cơ chế	Rò rỉ trong họ	Rò rỉ giữa các họ	Tôn trọng thời gian
Cách 1 — Random K-Fold	Xáo trộn ngẫu nhiên, chia 5 phần	Có	Có	Không
Cách 1b — Grouped K-Fold	Ngẫu nhiên, nhóm theo <code>origin_bar</code>	Không	Có	Không
Cách 2 — Walk-forward	Train quá khứ, test khúc kế tiếp	Không	Chỉ ở biên	Có
Cách 3 — Purged WF	Walk-forward + purge + embargo 50 bar	Không	Không	Có

4.3. Năm khúc thời gian và việc loại khúc 1

Dữ liệu được chia thành năm khúc bằng nhau theo thứ tự thời gian. Với walk-forward, fold đầu tiên huấn luyện trên khúc 1 và kiểm tra trên khúc 2; như vậy **khúc 1 không bao giờ có dự báo out-of-fold**, vì không có quá khứ nào để huấn luyện trước nó.

Để so sánh công bằng, chỉ số của cả bốn nhánh đều được **tính lại trên phạm vi khúc 2–5**, loại bỏ khúc 1 khỏi mọi nhánh — kể cả hai nhánh K-Fold vốn có dự báo cho khúc 1. Nếu không làm bước này, hai nhánh K-Fold sẽ được chấm trên nhiều dữ liệu hơn hai nhánh walk-forward và phép so sánh sẽ mất ý nghĩa.

Đây là chi tiết dễ bị bỏ qua nhất trong thiết kế. Mọi con số ở Bảng 5.1 và Bảng 5.2 đều được tính trên cùng một phạm vi khúc 2–5 gồm 20.007 lệnh.

4.4. Cấu hình mô hình

Bảng 4.2. Cấu hình CatBoost dùng chung cho cả bốn nhánh

Tham số	Giá trị	Loại
iterations	1000	Siêu tham số mô hình
learning_rate	0,05	Siêu tham số mô hình
depth	6	Siêu tham số mô hình
l2_leaf_reg	3,0	Siêu tham số mô hình
auto_class_weights	Balanced	Siêu tham số mô hình
eval_metric	AUC	Siêu tham số mô hình
random_seed	42	Thiết lập tái lập
thread_count	1	Thiết lập kỹ thuật (xem mục 5.6)
allow_writing_files	False	Thiết lập kỹ thuật

Cần phân biệt rõ hai loại tham số trong bảng trên. Sáu dòng đầu là **siêu tham số mô hình** — chúng định nghĩa mô hình. Ba dòng cuối là **thiết lập kỹ thuật** nhằm bảo đảm tái lập, không thuộc danh sách siêu tham số đã chốt ban đầu. Riêng `thread_count=1` được bổ sung về sau và trở thành chủ đề của một thí nghiệm riêng, trình bày ở mục 5.6.

4.5. Sinh và sử dụng xác suất out-of-fold

Với mỗi nhánh, mô hình sinh ra một bảng xác suất OOF: mỗi lệnh nhận đúng một xác suất, được dự báo bởi mô hình **không** được huấn luyện trên lệnh đó. Bảng xác suất này sau đó được dùng làm điểm số để lọc lệnh trong backtest. Cách làm này bảo đảm mỗi lệnh được chấm bởi một mô hình chưa từng nhìn thấy nó.

4.6. Quy tắc lọc top-k

Quy tắc lọc được chốt **trước khi mở holdout**: sắp xếp lệnh theo xác suất giảm dần (đồng hạng thì theo `row_id` tăng dần) và **giữ lại 50% lệnh điểm cao nhất**.

Báo cáo có trình bày bảng quét toàn dải 20–80 %, nhưng bảng này chỉ dùng để **phân tích độ nhạy**. Chúng tôi không chọn lại tỷ lệ giữ lệnh sau khi đã nhìn kết quả holdout. Nếu làm vậy, chính quy trình đánh giá sẽ tạo ra một tầng rò rỉ mới ở mức quyết định — đúng loại sai lầm mà đồ án đang nghiên cứu. Bailey và cộng sự [4] gọi hiện

tượng này là **backtest overfitting**: càng thử nhiều cấu hình rồi chọn cấu hình đẹp nhất, xác suất kết quả kiểm định bị thổi phồng càng cao.

4.7. Vấn đề tập lệnh ứng viên cố định

Cần ghi nhận trung thực một giới hạn của thiết kế backtest. Trong giao dịch thực, nếu bộ lọc bỏ một lệnh nền, các lệnh chồng phía sau sẽ không bao giờ được mở, và trạng thái của chiến lược sẽ khác đi — nghĩa là **việc lọc làm thay đổi chính tập tín hiệu phía sau**.

Để giữ cho bốn nhánh so sánh được với nhau, backtest duy trì một **chiến lược bóng (shadow strategy)**: chiến lược gốc vẫn chạy đầy đủ để sinh tập lệnh ứng viên cố định, còn bộ lọc chỉ quyết định lệnh nào được tính vào kết quả. Hệ quả là các con số tài chính trong báo cáo nên được hiểu là **ước lượng cận trên**, không phải mô phỏng giao dịch trực tiếp.

4.8. Quy trình holdout năm giai đoạn

- **Giai đoạn 1 — Tách.** Sinh lại toàn bộ lịch sử và tách tập holdout; đối chiếu để chắc chắn phần trước mốc trùng khớp với tập đã đóng băng.
- **Giai đoạn 2 — Huấn luyện.** Huấn luyện mô hình cuối hai lần độc lập và kiểm tra hai lần cho dự báo giống hệt nhau.
- **Giai đoạn 3 — Chấm điểm và backtest.** Chấm điểm 5.028 lệnh holdout và backtest theo từng mức giữ lệnh.
- **Giai đoạn 4 — Lập bảng và biểu đồ.** Sinh bảng kết quả và đường equity; bước này có kiểm tra fail-closed, chỉ xuất báo cáo khi các cờ bằng chứng ở giai đoạn 1–3 đều đạt.
- **Giai đoạn 5 — Đối chiếu.** So sánh toàn chuỗi giữa hai lần chạy độc lập.

4.9. Biện pháp bảo đảm tái lập

- Ghim `random_seed=42` và `thread_count=1`.
- So sánh **từng byte** giữa hai lần chạy độc lập của cả chuỗi huấn luyện và backtest.
- Niêm phong artifact bàn giao: các thư mục kết quả gốc được giữ nguyên, mọi lần chạy lại ghi sang cây thư mục mới.

- Bộ kiểm thử tự động gồm 13 test kiểm tra hằng số, cấu trúc gói và tính nhất quán của bằng chứng.

CHƯƠNG 5. KẾT QUẢ THỬ NGHIỆM

Toàn bộ số liệu trong chương này lấy từ cây kết quả chuẩn outputs/step4_thread1/ (bốn nhánh) và outputs/holdout/ (holdout niêm phong).

Quy ước số: báo cáo dùng dấu phẩy làm dấu thập phân và dấu chấm phân nhóm nghìn theo chuẩn Việt Nam (ví dụ: 0,8582 và 25.008). Riêng tên tệp, tên tham số và giá trị trích từ mã nguồn được giữ nguyên định dạng gốc (ví dụ: learning_rate=0.05).

5.1. Chỉ số phân loại của bốn cách chia

Bảng 5.1. Chỉ số phân loại của bốn cách chia và của holdout

Cách chia	ROC-AUC	F1 @ 0,5
Cách 1 — Random K-Fold	0,8582	0,6494
Cách 1b — Grouped K-Fold	0,7454	0,5077
Cách 2 — Walk-forward	0,5875	0,3288
Cách 3 — WF + Purge/Embargo	0,5895	0,3247
Holdout niêm phong	0,6046	0,4022

Nguồn: outputs/holdout/stage4/table1_classification_metrics.csv. Bốn dòng đầu là trung bình theo fold trên phạm vi khúc 2–5; dòng holdout tính một lần trên toàn bộ 5.028 mẫu.

Đây là kết quả trung tâm của đồ án. Ba quan sát:

1. **Suy giảm đơn điệu theo độ chặt.** ROC-AUC giảm liên tục khi cách chia siết chặt dần: $0,8582 \rightarrow 0,7454 \rightarrow 0,5875 \rightarrow 0,5895$. Mỗi lần chặn thêm một tầng rò rỉ, điểm số lại tụt xuống.
2. **Holdout nằm đúng ở mức walk-forward.** Giá trị holdout 0,6046 gần như trùng với Cách 2 (0,5875) và Cách 3 (0,5895), chênh chưa tới 0,02 đơn vị, trong khi cách Random K-Fold (0,8582) tới **0,25 đơn vị**.
3. **Định lượng được mức thổi phồng.** Chênh lệch giữa ước lượng của Random K-Fold và thực tế holdout là **+0,2536 ROC-AUC** — gần bằng toàn bộ khoảng cách từ một mô hình đoán mò (0,5) tới một mô hình rất tốt (0,85).

Chỉ số F1 xác nhận cùng một xu hướng: $0,6494 \rightarrow 0,5077 \rightarrow 0,3288 \rightarrow 0,3247$, với holdout ở mức 0,4022. Đáng chú ý là F1 trên holdout (0,4022) **cao hơn** cả hai nhánh

walk-forward, trong khi ROC-AUC cũng cao hơn chút ít; điều này phản ánh tập holdout là một giai đoạn thị trường khác, không phải mô hình tốt lên.

5.2. Chỉ số tài chính khi giữ top 50%

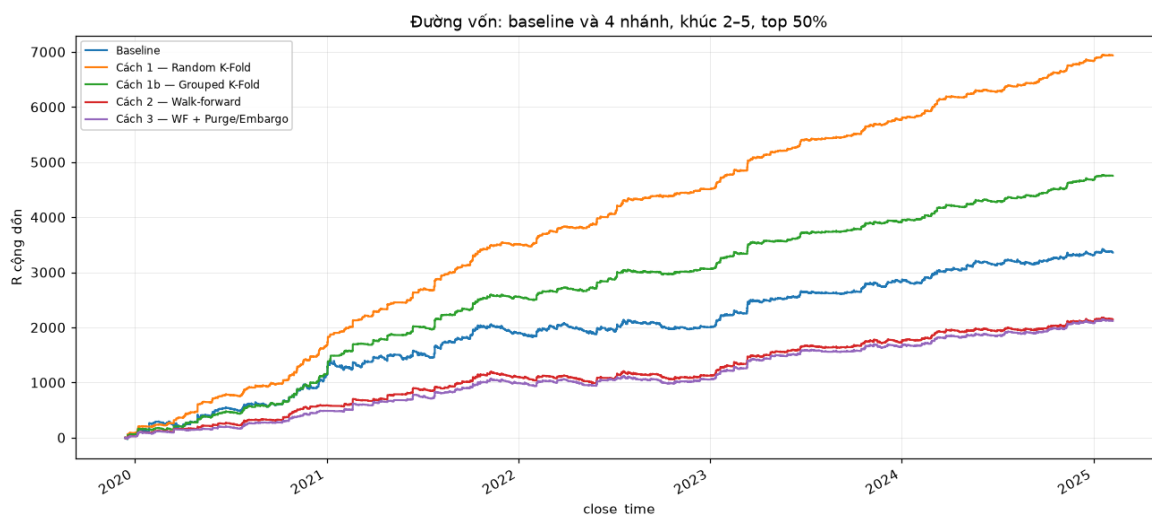
Bảng 5.2. Chỉ số tài chính khi giữ top 50% trên khúc 2–5

	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Baseline (không lọc)	20.007	+3.358,25	236,13	1,2940	31,71 %
Cách 1 — Random K-Fold	10.004	+6.937,53	73,76	2,5437	44,65 %
Cách 1b — Grouped K-Fold	10.004	+4.752,04	99,71	1,9500	39,26 %
Cách 2 — Walk-forward	10.004	+2.148,31	211,29	1,4117	34,76 %
Cách 3 — WF + Purge/Embargo	10.004	+2.119,48	142,36	1,4074	34,99 %

Nguồn: *outputs/holdout/stage4/table2_financial_metrics_top50.csv*

Trên chỉ số tài chính, hậu quả của việc chọn sai cách chia còn nghiêm trọng hơn. Cùng một mô hình, cùng một tập lệnh, cùng một tỷ lệ lọc 50 %, nhưng net profit chênh nhau **hơn ba lần**: +6.937,53 R ở nhánh Random K-Fold so với +2.119,48 R ở nhánh Purged Walk-forward.

Khoảng cách ở MaxDD còn rõ hơn: 73,76 R so với 142,36 R — gấp gần hai lần. Và profit factor 2,5437 so với 1,4074 là chênh lệch giữa **một chiến lược trông xuất sắc** và **một chiến lược chỉ nhỉnh hơn baseline một chút**. Một nhóm nghiên cứu chỉ nhìn dòng đầu tiên sẽ đưa ra kết luận kinh doanh hoàn toàn khác so với nhóm nhìn dòng cuối.



Hình 5.1. Đường equity của bốn nhánh trên khúc 2–5 khi giữ top 50%

5.3. Độ nhạy theo tỷ lệ giữ lệnh của bốn nhánh

Bảng 5.3. Trích bảng sweep tỷ lệ giữ lệnh của bốn nhánh

Cách chia	Mức giữ	Số lệnh	Net profit (R)	PF	Win rate
Cách 1 — Random K-Fold	Top 20 %	4.002	+5.248,16	6,3257	64,82 %
Cách 1 — Random K-Fold	Top 50 %	10.004	+6.937,53	2,5437	44,65 %
Cách 1 — Random K-Fold	Top 80 %	16.006	+4.980,00	1,5665	34,61 %
Cách 1b — Grouped K-Fold	Top 20 %	4.002	+2.803,94	2,7979	48,23 %
Cách 1b — Grouped K-Fold	Top 50 %	10.004	+4.752,04	1,9500	39,26 %
Cách 1b — Grouped K-Fold	Top 80 %	16.006	+4.297,70	1,4817	33,63 %
Cách 2 — Walk-forward	Top 20 %	4.002	+1.056,60	1,5463	37,21 %
Cách 2 — Walk-forward	Top 50 %	10.004	+2.148,31	1,4117	34,76 %
Cách 2 — Walk-forward	Top 80 %	16.006	+3.120,69	1,3581	33,50 %
Cách 3 — WF + Purge/Embargo	Top 20 %	4.002	+1.266,26	1,6598	38,26 %
Cách 3 — WF + Purge/Embargo	Top 50 %	10.004	+2.119,48	1,4074	34,99 %
Cách 3 — WF + Purge/Embargo	Top 80 %	16.006	+3.061,29	1,3510	33,38 %

Nguồn: *outputs/holdout/stage4/table3_branch_sweep_20_80.csv* (bảng đầy đủ 28 dòng ở Phụ lục B)

Bảng này bộc lộ một khác biệt về **hình dạng** chứ không chỉ về độ lớn. Ở hai nhánh có rò rỉ (Cách 1 và 1b), profit factor tăng rất mạnh khi lọc gắt hơn — Cách 1 đạt PF 6,3257 và win rate 64,82 % ở mức Top 20 %. Ở hai nhánh walk-forward, lọc gắt hơn lại **làm giảm** net profit (+1.056,60 R ở Top 20 % so với +3.120,69 R ở Top 80 %) và PF gần như đi ngang quanh 1,35–1,66.

Nói cách khác: ở nhánh rò rỉ, điểm số cao của mô hình thực sự chỉ ra những lệnh thắng — vì mô hình đã biết đáp án. Ở nhánh không rò rỉ, năng lực xếp hạng yếu đi rất nhiều và lợi ích của việc lọc gần như biến mất.

5.4. Kết quả trên tập holdout niêm phong

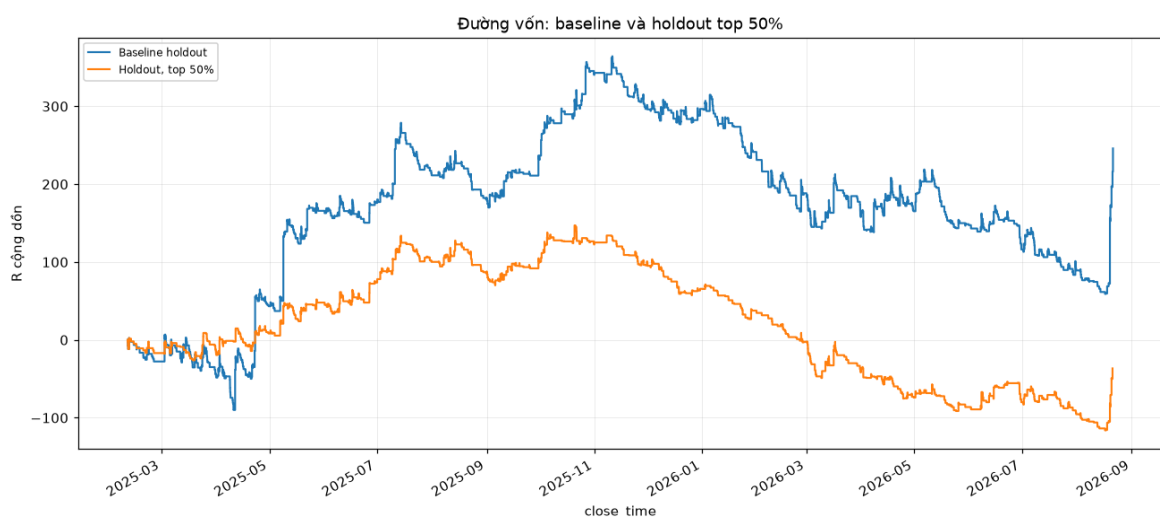
Đây là phép thử trọng tài: tập holdout 5.028 lệnh, trải từ 2025-02-08 tới 2026-08-21, được chấm điểm đúng một lần bằng mô hình cuối.

Bảng 5.4. Kết quả holdout niêm phong khi giữ top 50%

	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Baseline holdout (không lọc)	5.028	+245,93	305,32	1,0992	35,28 %
Holdout, giữ top 50 %	2.514	-36,55	263,25	0,9718	34,81 %

Nguồn: `outputs/holdout/stage3/stage3_report.json`

Trên dữ liệu chưa từng được nhìn, bộ lọc **làm xấu đi** kết quả: từ +245,93 R xuống -36,55 R, và profit factor tụt từ 1,0992 xuống 0,9718 — tức dưới ngưỡng hòa vốn. Việc dùng mô hình để lọc tín hiệu, trong thiết lập này, **kém hơn việc không lọc gì cả**.



Hình 5.2. Đường equity trên tập holdout niêm phong khi giữ top 50%

5.5. Độ nhảy theo tỷ lệ giữ lệnh trên holdout

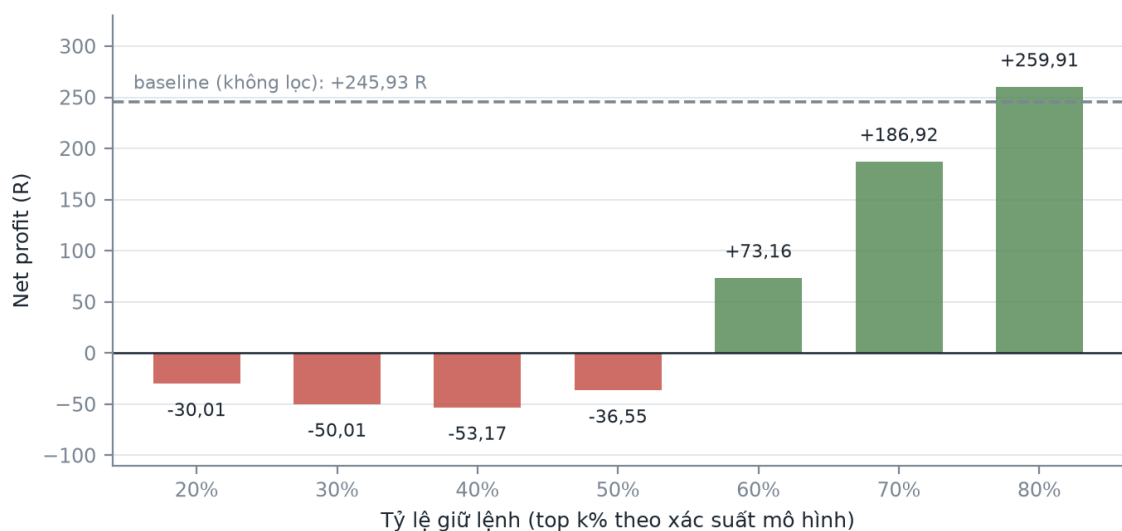
Một phản biện tự nhiên là: có lẽ chỉ do chọn nhầm tỷ lệ giữ lệnh. Bảng quét toàn dải bác bỏ giả thuyết đó, và còn cho thấy điều nghiêm trọng hơn.

Bảng 5.5. Sweep tỷ lệ giữ lệnh trên tập holdout

Mức giữ	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Top 20 %	1.006	-30,01	125,38	0,9385	36,18 %

Mức giữ	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Top 30 %	1.509	-50,01	210,77	0,9343	35,52 %
Top 40 %	2.012	-53,17	251,56	0,9485	35,04 %
Top 50 %	2.514	-36,55	263,25	0,9718	34,81 %
Top 60 %	3.017	+73,16	233,95	1,0474	35,60 %
Top 70 %	3.520	+186,92	197,24	1,1040	35,77 %
Top 80 %	4.023	+259,91	219,02	1,1278	35,52 %
Baseline (100 %)	5.028	+245,93	305,32	1,0992	35,28 %

Nguồn: *outputs/holdout/stage4/holdout_sweep_20_80.csv*



Hình 5.3. Net profit trên holdout theo từng mức tỷ lệ giữ lệnh

Quy luật trong bảng này là điều đáng chú ý nhất của toàn bộ kết quả holdout: **lọc càng gắt, kết quả càng xấu**. Bốn mức lọc chặt nhất đều âm, và chỉ từ Top 60 % trở đi net profit mới dương.

Điều đó có nghĩa là thứ hạng xác suất của mô hình **bị đảo chiều** trên dữ liệu mới: nhóm lệnh mà mô hình chấm điểm cao nhất lại cho kết quả kém nhất. Đây là bằng chứng mạnh hơn nhiều so với một kết quả âm đơn thuần — nó cho thấy mô hình không chỉ mất năng lực dự báo mà còn **xếp hạng sai hướng**.

Cũng cần đọc đúng các mức dương: Top 80 % đạt +259,91 R, nhỉnh hơn baseline +245,93 R, nhưng ở mức đó bộ lọc chỉ loại 20 % số lệnh, nên khác biệt so với việc không lọc là rất nhỏ và không thể coi là bằng chứng mô hình có giá trị.

5.6. Kiểm chứng tái lập

Bảng 5.6. Kết quả kiểm chứng tái lập

Hạng mục kiểm chứng	Kết quả
Khóa dòng của tập sinh lại so với tập đóng băng	25.008 / 25.008 trùng khớp
Ô đặc trưng khớp chính xác	575.184 / 575.184
Số dòng bị loại bởi purge và embargo ở giai đoạn holdout	0
Hai lượt huấn luyện độc lập cho dự báo giống nhau	Đạt — sai lệch tối đa 0,0
Backtest baseline khớp số lệnh holdout	5.028 / 5.028
Đối chiếu toàn chuỗi giữa hai lần chạy (giai đoạn 3–4)	PASS — mọi sai lệch 0,0; cả 4 biểu đồ trùng từng byte
Bộ kiểm thử tự động	13 / 13 đạt

Nguồn: `outputs/holdout/repro/stage5_repro_report.json`; `outputs/holdout/stage1–stage3`

Cần nêu rõ giới hạn: các kiểm chứng trên đều là **cùng một máy**. Tính bằng nhau giữa các máy khác nhau chưa được chứng minh. Một thành viên khác đã chạy lại trên môi trường Windows và báo rằng dự báo lệch trong khoảng $1e-15$, nhưng **không byte-identical** và không kèm số liệu chỉ số nào, nên đây chỉ là thông tin tham khảo.

5.7. Thí nghiệm có kiểm soát về tham số `thread_count`

Trong quá trình kiểm chứng, nhóm phát hiện kết quả huấn luyện thay đổi giữa các lần chạy dù đã ghim `random_seed`. Giả thuyết ban đầu là do số luồng thực thi khác nhau. Để kiểm chứng, một thí nghiệm có kiểm soát được dựng: **chỉ thay đổi duy nhất `thread_count`**, giữ nguyên mọi thứ khác.

Bảng 5.7. Ảnh hưởng của `thread_count` tới kết quả huấn luyện

Cấu hình	$\max \Delta p $ tập train	$\max \Delta p $ holdout	Tương quan Pearson	Net R holdout top 50 %
<code>thread_count=1</code> , lần chạy 2	0	0	1,0000	−36,55
<code>thread_count=2</code>	0,2832	0,3469	0,9331	−12,26
<code>thread_count=-1</code> (mặc định)	0,2999	0,3352	0,9409	+30,79

Nguồn: `outputs/thread_count_sensitivity/thread_count_sensitivity_summary.csv`. So sánh với lần chạy tham chiếu `thread_count=1`.

Ba kết luận rút ra:

1. Hai lần chạy cùng cấu hình `thread_count=1` cho kết quả **trùng khớp từng bit**.
2. Đổi `thread_count` sang 2 hoặc để mặc định làm thay đổi **toàn bộ 25.008 dự báo** trên tập train và **toàn bộ 5.028 dự báo** trên holdout.
3. Hiệu ứng nằm ở **khâu huấn luyện, không phải khâu suy luận**: chấm điểm lại cùng một mô hình đã huấn luyện ở các mức `thread_count` khác nhau cho xác suất giống hệt nhau.

Ý nghĩa thực tiễn của phát hiện này rất đáng lưu ý: nếu không ghim `thread_count`, cùng một mã nguồn và cùng một hạt giống vẫn cho kết luận holdout trái ngược nhau — từ **-36,55 R** thành **+30,79 R**, tức từ thua lỗ thành có lãi.

Tài liệu chính thức của CatBoost [5] mô tả `thread_count` là tham số chỉ ảnh hưởng tốc độ: *“Optimizes the speed of execution. This parameter doesn't affect results.”* Phép đo của nhóm **xác nhận điều đó ở khâu suy luận nhưng mâu thuẫn ở khâu huấn luyện**. Phạm vi kết luận giới hạn ở một máy, một bản build CatBoost, một tập dữ liệu và một hạt giống; đây không phải cơ sở để quy toàn bộ sai lệch giữa các máy cho tham số này.

CHƯƠNG 6. BÀN LUẬN

6.1. Trả lời các câu hỏi nghiên cứu

6.1.1. RQ1 — Mức ảnh hưởng của cách chia dữ liệu

Ảnh hưởng là rất lớn và đo được. Trên chỉ số phân loại, khoảng cách giữa cách chia lỏng nhất và chặt nhất là **0,2687 ROC-AUC** (0,8582 so với 0,5895). Trên chỉ số tài chính, net profit chênh **hơn ba lần** và profit factor chênh từ 2,5437 xuống 1,4074. Vì mọi yếu tố khác đã được giữ nguyên, toàn bộ khoảng cách này quy về đúng một nguyên nhân là cách chia dữ liệu.

6.1.2. RQ2 — Sự hội tụ khi siết chặt cách chia

Có hội tụ rõ rệt. Bước từ Random K-Fold sang Grouped K-Fold làm giảm 0,1128 ROC-AUC; bước từ Grouped K-Fold sang Walk-forward giảm tiếp 0,1579; nhưng bước từ Walk-forward sang Purged Walk-forward **gần như không đổi** (0,5875 so với 0,5895, chênh 0,0020 và thậm chí đổi dấu). Điều này cho thấy hai nhánh walk-forward đã chạm tới mức ổn định.

6.1.3. RQ3 — Cách chia nào dự báo đúng nhất

Bảng 6.1. So sánh ước lượng của từng cách chia với holdout

Cách chia	ROC-AUC ước lượng	Sai lệch so với holdout (0,6046)
Cách 1 — Random K-Fold	0,8582	+0,2536
Cách 1b — Grouped K-Fold	0,7454	+0,1408
Cách 2 — Walk-forward	0,5875	−0,0171
Cách 3 — WF + Purge/Embargo	0,5895	−0,0151

Hai nhánh walk-forward cho ước lượng sát holdout nhất, sai lệch chưa tới 0,02 đơn vị. Random K-Fold sai lệch gấp **khoảng 15 lần** so với walk-forward. Đây là câu trả lời trực tiếp cho RQ3: với dữ liệu có nhãn chồng lấn, walk-forward là mức tối thiểu cần thiết, còn Random K-Fold không dùng được.

6.2. Vì sao Grouped K-Fold nằm ở giữa

Grouped K-Fold chặn được tầng rò rỉ thứ nhất — các leg cùng họ không còn bị tách hai phía — nên điểm số tụt 0,1128 so với Random K-Fold. Nhưng nó vẫn để lọt

tầng thứ hai: hai họ lệnh mở cách nhau vài chục bar vẫn có cửa sổ nhân giao nhau, và việc xáo trộn ngẫu nhiên vẫn đặt chúng ở hai phía khác nhau.

Ngoài ra Grouped K-Fold **không hề tôn trọng mũi tên thời gian**: mô hình vẫn được học trên dữ liệu từ 2024 để dự báo cho 2019. Với dữ liệu không dừng như đã thấy ở Bảng 3.4, riêng điều này đã đủ tạo ra ưu thế không thực tế. Vị trí trung gian 0,7454 của nhánh này phản ánh đúng việc nó **xử lý được một nửa vấn đề**.

6.3. Kết quả âm ngoài dự kiến: purge và embargo gần như không có tác dụng

Giả thuyết ban đầu là Cách 3 sẽ cho điểm số thấp hơn Cách 2 một cách rõ ràng, vì nó chặt hơn. Thực tế hai nhánh gần như bằng nhau, và Cách 3 thậm chí **cao hơn** một chút (0,5895 so với 0,5875) — nằm trong biên nhiễu.

Nguyên nhân có thể giải thích bằng số học:

- Cửa sổ nhân dài tối đa $H = 50$ bar, tương ứng một số ít dòng dữ liệu quanh mỗi biên fold.
- Mỗi khúc chứa khoảng 5.000 dòng, nên vùng chồng lấn ở biên chiếm **chưa tới 1 %** dữ liệu huấn luyện.
- Trên thực tế, purge và embargo chỉ loại **21, 3, 8 và 2 dòng** ở bốn biên fold tương ứng.

Loại bỏ vài chục dòng trên tổng số hai mươi nghìn dòng thì không thể làm đổi chỉ số một cách có ý nghĩa. Đây là một **kết quả âm** và chúng tôi trình bày đúng như vậy: trong thiết lập này, bản thân walk-forward đã xử lý gần hết vấn đề rò rỉ, còn purge và embargo là lớp bảo hiểm bổ sung với chi phí gần như bằng không nhưng cũng không mang lại cải thiện đo được.

Kết luận này **phụ thuộc vào tỷ lệ giữa H và độ dài khúc**. Với bài toán có cửa sổ nhân dài hơn nhiều so với khúc dữ liệu, purge và embargo sẽ có vai trò lớn hơn hẳn. Không nên suy rộng kết quả này thành 'purge/embargo là không cần thiết'.

6.4. Ý nghĩa thực tiễn

Nếu nhóm chỉ thực hiện Cách 1 — vốn là lựa chọn mặc định trong hầu hết giáo trình học máy — thì kết luận sẽ là: mô hình lọc tín hiệu rất tốt, ROC-AUC 0,8582, profit

factor 2,5437, net profit gấp đôi baseline. Trên thực tế, holdout cho thấy mô hình **làm xấu kết quả**.

Khoảng cách giữa hai kết luận đó chính là chi phí của một quyết định phương pháp luận duy nhất. Trong bối cảnh ứng dụng thật, chi phí này không dừng ở điểm số: nó là vốn thật bị đưa vào một hệ thống được tin tưởng nhầm.

Biện pháp phòng vệ hiệu quả nhất trong đề án này lại rất rẻ: **tách một tập holdout trước khi bắt đầu và không đụng tới nó**. Tập holdout không đòi hỏi kỹ thuật phức tạp, chỉ đòi hỏi kỷ luật.

6.5. Đối chiếu với tài liệu

Vấn đề rò rỉ nhãn do chồng lấn thời gian, kiến trúc meta-labeling và hai kỹ thuật purging, embargo được trình bày hệ thống trong López de Prado [1]. Kết quả của đề án **xác nhận luận điểm cốt lõi** của tài liệu này về mức nguy hiểm của kiểm định chéo ngẫu nhiên trên dữ liệu tài chính. Bergmeir và Benítez [3] cũng đã chỉ ra trên chuỗi thời gian tổng quát rằng kiểm định chéo ngẫu nhiên cho ước lượng thiên lệch khi dữ liệu có phụ thuộc chuỗi, và đề xuất dạng kiểm định chéo theo khối — cùng hướng với kết quả đo được ở Bảng 5.1.

Tuy nhiên có một điểm khác biệt đáng ghi nhận: trong thiết lập của đề án, phần lớn lợi ích đến từ **việc chuyển sang walk-forward**, còn purging và embargo đóng góp không đáng kể. Điều này nhất quán với phân tích ở mục 6.3 và cho thấy mức độ cần thiết của từng kỹ thuật phụ thuộc vào tỷ lệ giữa độ dài cửa sổ nhãn và độ dài khúc dữ liệu, chứ không phải là một quy tắc áp dụng đồng đều.

CHƯƠNG 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1. Hạn chế

1. **Phạm vi hẹp.** Kết quả dựa trên một tài sản (BTCUSD), một khung thời gian (M15) và một chiến lược (pyramid $k = 3$). Chưa thể khẳng định mức thiên lệch 0,25 ROC-AUC là điển hình cho các bài toán khác.
2. **Tập lệnh ứng viên cố định.** Backtest dùng chiến lược bóng để giữ tập lệnh không đổi, nên các con số tài chính là **ước lượng cận trên**, không phải mô phỏng giao dịch trực tiếp (mục 4.7).
3. **Chưa tính chi phí giao dịch.** Backtest không tính phí, trượt giá và chi phí qua đêm. Với 20.007 lệnh, các chi phí này sẽ làm giảm đáng kể mọi con số net profit.
4. **Một hạt giống, một cấu hình.** Mọi kết quả dùng `random_seed=42` và một bộ siêu tham số duy nhất. Chưa có ước lượng phương sai giữa các hạt giống, nên chênh lệch nhỏ như 0,0020 giữa Cách 2 và Cách 3 không thể diễn giải chắc chắn.
5. **Tái lập giữa các máy chưa được chứng minh.** Chỉ có bằng chứng trên cùng một máy. Kết quả xác minh của thành viên khác trên Windows lệch trong $1e-15$ nhưng không byte-identical và không kèm số liệu.
6. **Lịch sử mở holdout không đầy đủ.** Sổ lịch sử dựa trên ba commit truy xuất được từ Git, không phải nhật ký toàn bộ mọi lần thực thi. Đồ án không chứng minh được rằng không có quyết định thiết kế nào chịu ảnh hưởng từ việc đã nhìn holdout.

7.2. Hướng phát triển

- **Ước lượng phương sai.** Chạy lại toàn bộ thí nghiệm với nhiều hạt giống để có khoảng tin cậy cho từng chỉ số, từ đó xác định được chênh lệch nào là thật và chênh lệch nào nằm trong nhiễu.
- **Combinatorial Purged Cross-Validation.** Áp dụng CPCV để có nhiều đường kiểm định thay vì một chuỗi walk-forward duy nhất, giúp ước lượng ổn định hơn.
- **Mở rộng phạm vi.** Lặp lại thí nghiệm trên tài sản khác, khung thời gian khác và cửa sổ nhãn dài hơn — đặc biệt là trường hợp H lớn so với độ dài khúc, nơi purge và embargo được dự đoán sẽ có vai trò rõ rệt.

- **Đưa chi phí giao dịch vào backtest.** Bổ sung phí, trượt giá và chi phí qua đêm để các con số tài chính phản ánh sát điều kiện thực.
- **Hiệu chuẩn xác suất.** Thay quy tắc top-k cứng bằng ngưỡng dựa trên xác suất đã hiệu chuẩn, cho phép số lệnh giữ lại thay đổi theo mức độ tin cậy thực tế.
- **Kiểm tra tái lập đa nền tảng.** Chạy toàn bộ chuỗi trên nhiều hệ điều hành và nhiều bản build CatBoost để xác định phạm vi hiệu lực của phát hiện về `thread_count`.

CHƯƠNG 8. KẾT LUẬN

Đề án đặt ra và trả lời bằng số liệu một câu hỏi phương pháp luận: cách chia dữ liệu ảnh hưởng bao nhiêu tới đánh giá mô hình học máy trên chuỗi thời gian tài chính có nhãn chồng lấn.

Thứ nhất, ảnh hưởng là rất lớn và đo được. Với mọi yếu tố khác giữ nguyên, việc chuyển từ Random K-Fold sang Purged Walk-forward làm ROC-AUC giảm 0,2687 đơn vị và net profit giảm hơn ba lần. Cách chia dữ liệu không phải một chi tiết kỹ thuật thứ yếu — nó là biến quyết định kết luận.

Thứ hai, Random K-Fold không dùng được cho lớp bài toán này. Ước lượng của nó lệch khỏi thực tế holdout tới +0,2536 ROC-AUC, trong khi walk-forward chỉ lệch -0,0171 — sai số nhỏ hơn khoảng 15 lần. Trong thiết lập của đề án, phần lớn lợi ích đến từ việc tôn trọng mũi tên thời gian; purge và embargo chỉ loại 34 dòng trên tổng số hơn hai mươi nghìn và không tạo khác biệt đo được.

Thứ ba, holdout niêm phong cho thấy bộ lọc hiện tại chưa tạo ra giá trị. Trên dữ liệu chưa từng nhìn, việc lọc top 50 % cho -36,55 R so với +245,93 R khi không lọc, và bảng quét toàn dải cho thấy thứ hạng của mô hình còn bị đảo chiều: lọc càng gắt càng lỗ.

Đóng góp của đề án vì vậy không nằm ở một con số lợi nhuận, mà ở **một quy trình đánh giá trung thực và tái lập được**: tách holdout trước khi bắt đầu, so sánh có kiểm soát chỉ đổi một biến, công bố cả kết quả âm, và kèm bằng chứng kiểm chứng từng byte cho mỗi con số được báo cáo. Một mô hình cho kết quả âm nhưng được đo đúng có giá trị hơn nhiều so với một mô hình cho kết quả đẹp nhưng được đo sai.

TÀI LIỆU THAM KHẢO

- [1] López de Prado, M. (2018). *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons. ISBN 978-1-119-48208-6.
- [2] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. arXiv:1706.09516.
- [3] Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213. DOI: 10.1016/j.ins.2011.12.028.
- [4] Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5), tr. 458.
- [5] Yandex. *CatBoost Documentation — Training parameters: Performance settings (thread_count)*. <https://catboost.ai/docs/en/references/training-parameters/performance>. Truy cập tháng 9 năm 2026.

PHỤ LỤC A. DANH SÁCH ĐẦY ĐỦ 23 ĐẶC TRƯNG

STT	Tên đặc trưng	Nhóm	Mô tả
1	breakeven_R	A	Khoảng cách tới điểm hòa vốn tính theo bội số R
2	atr14_pct	B	ATR chu kỳ 14, biểu diễn theo phần trăm giá
3	atr_ratio_14_90	B	Tỷ lệ ATR(14) trên ATR(90) — biến động ngắn so với dài hạn
4	vol20	B	Độ lệch chuẩn lợi suất 20 bar
5	vol200	B	Độ lệch chuẩn lợi suất 200 bar
6	vol_ratio_20_200	B	Tỷ lệ giữa hai mức biến động trên
7	range_pct	B	Biên độ bar hiện tại theo phần trăm
8	dist_ema20_atr	C	Khoảng cách tới EMA(20), chuẩn hóa theo ATR
9	dist_ema50_atr	C	Khoảng cách tới EMA(50), chuẩn hóa theo ATR
10	dist_ema200_atr	C	Khoảng cách tới EMA(200), chuẩn hóa theo ATR
11	ret20_atr	C	Lợi suất 20 bar, chuẩn hóa theo ATR
12	ret50_atr	C	Lợi suất 50 bar, chuẩn hóa theo ATR
13	pos_in_range50	C	Vị trí giá trong biên độ 50 bar, thang [0, 1]
14	dist_hh20_atr	C	Khoảng cách tới đỉnh cao nhất 20 bar, chuẩn hóa theo ATR
15	mom14	D	Động lượng chu kỳ 14
16	mom_diff	D	Sai phân động lượng
17	vwap_dist_atr	D	Khoảng cách tới VWAP, chuẩn hóa theo ATR
18	vwap_slope_atr	D	Độ dốc VWAP, chuẩn hóa theo ATR
19	rsi14	D	Chỉ số RSI chu kỳ 14, thang [0, 100]
20	vol_ratio_volume	E	Tỷ lệ khối lượng hiện tại so với trung bình
21	gap_open_atr	E	Khoảng nhảy giá mở cửa, chuẩn hóa theo ATR
22	hour	E	Giờ trong ngày, thang 0–23
23	dow	E	Thứ trong tuần, thang 0–6

PHỤ LỤC B. BẢNG SWEEP ĐẦY ĐỦ CỦA BỐN NHÁNH

Baseline khúc 2–5 (không lọc): 20.007 lệnh, +3.358,25 R, MaxDD 236,13 R, PF 1,2940, win rate 31,71 %.

Cách chia	Mức giữ	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Cách 1 — Random K-Fold	Top 20 %	4.002	+5.248,16	13,49	6,3257	64,82 %
Cách 1 — Random K-Fold	Top 30 %	6.003	+6.640,38	23,91	4,4036	57,02 %
Cách 1 — Random K-Fold	Top 40 %	8.003	+7.136,31	34,59	3,2781	50,58 %
Cách 1 — Random K-Fold	Top 50 %	10.004	+6.937,53	73,76	2,5437	44,65 %
Cách 1 — Random K-Fold	Top 60 %	12.005	+6.451,10	110,82	2,0872	40,32 %
Cách 1 — Random K-Fold	Top 70 %	14.005	+5.800,28	136,66	1,7880	37,14 %
Cách 1 — Random K-Fold	Top 80 %	16.006	+4.980,00	178,18	1,5665	34,61 %
Cách 1b — Grouped K-Fold	Top 20 %	4.002	+2.803,94	33,27	2,7979	48,23 %
Cách 1b — Grouped K-Fold	Top 30 %	6.003	+3.696,16	47,29	2,4148	44,76 %
Cách 1b — Grouped K-Fold	Top 40 %	8.003	+4.325,22	67,22	2,1492	41,83 %
Cách 1b — Grouped K-Fold	Top 50 %	10.004	+4.752,04	99,71	1,9500	39,26 %
Cách 1b — Grouped K-Fold	Top 60 %	12.005	+4.772,24	150,80	1,7581	37,08 %
Cách 1b — Grouped K-Fold	Top 70 %	14.005	+4.521,23	191,83	1,5925	35,08 %
Cách 1b — Grouped K-Fold	Top 80 %	16.006	+4.297,70	193,95	1,4817	33,63 %
Cách 2 — Walk-forward	Top 20 %	4.002	+1.056,60	103,86	1,5463	37,21 %
Cách 2 — Walk-forward	Top 30 %	6.003	+1.327,33	159,63	1,4409	36,27 %

Cách chia	Mức giữ	Số lệnh	Net profit (R)	MaxDD (R)	PF	Win rate
Cách 2 — Walk-forward	Top 40 %	8.003	+1.695,46	157,93	1,4115	35,34 %
Cách 2 — Walk-forward	Top 50 %	10.004	+2.148,31	211,29	1,4117	34,76 %
Cách 2 — Walk-forward	Top 60 %	12.005	+2.575,06	240,18	1,4051	34,37 %
Cách 2 — Walk-forward	Top 70 %	14.005	+2.760,58	283,51	1,3664	33,75 %
Cách 2 — Walk-forward	Top 80 %	16.006	+3.120,69	273,52	1,3581	33,50 %
Cách 3 — WF + Purge/Embargo	Top 20 %	4.002	+1.266,26	86,49	1,6598	38,26 %
Cách 3 — WF + Purge/Embargo	Top 30 %	6.003	+1.381,02	149,36	1,4623	36,40 %
Cách 3 — WF + Purge/Embargo	Top 40 %	8.003	+1.674,66	144,96	1,4107	35,67 %
Cách 3 — WF + Purge/Embargo	Top 50 %	10.004	+2.119,48	142,36	1,4074	34,99 %
Cách 3 — WF + Purge/Embargo	Top 60 %	12.005	+2.323,85	208,79	1,3642	34,12 %
Cách 3 — WF + Purge/Embargo	Top 70 %	14.005	+2.688,96	221,19	1,3578	33,77 %
Cách 3 — WF + Purge/Embargo	Top 80 %	16.006	+3.061,29	240,51	1,3510	33,38 %

Nguồn: *outputs/holdout/stage4/table3_branch_sweep_20_80.csv*

PHỤ LỤC C. LỊCH SỬ BA PHIÊN BẢN KẾT QUẢ HOLDOUT

Ba phiên bản kết quả holdout truy xuất được từ lịch sử Git được ghi lại để bảo đảm minh bạch. Đây **không phải nhật ký đầy đủ** của mọi lần thực thi; sổ chỉ ghi ba commit đã xác định được.

Phiên bản	Commit	Môi trường	thread_count	ROC-AUC	F1	Top 50 % net R
Lần 1 — Windows gốc	dc25cd3	Windows	không ghim	0,6050	0,4017	−23,83
Lần 2 — Linux đa luồng	5f46e41	Linux	không ghim	0,6023	0,4065	+30,79
Lần 3 — artifact hiện tại	7748828	Linux	1	0,6046	0,4022	−36,55

Nguồn: *outputs/verification/holdout_run_history.json*

Baseline holdout giống nhau ở cả ba lần (5.028 lệnh, +245,93 R), vì baseline là tradelist tĩnh không phụ thuộc mô hình. Lý do chạy lại: cấu hình CatBoost ban đầu chưa ghim thread_count; lần 3 chỉ bổ sung thiết lập kỹ thuật này, **không thay đổi** đặc trưng, siêu tham số mô hình, tập huấn luyện, tập holdout hay quy tắc lọc top-k.